

The Observer Pattern in Unity

- player destroys an enemy
Player ----> Enemy
Player communicates damage to the enemy.
- player collects a power-up
PowerUp ----> Player
PowerUp communicates stat changes to the Player
- player reaches a new level
Level -----> Player
Level communicates stat changes to the Player.

You need a way for objects to notify other objects.
But you can't always directly reference them together.

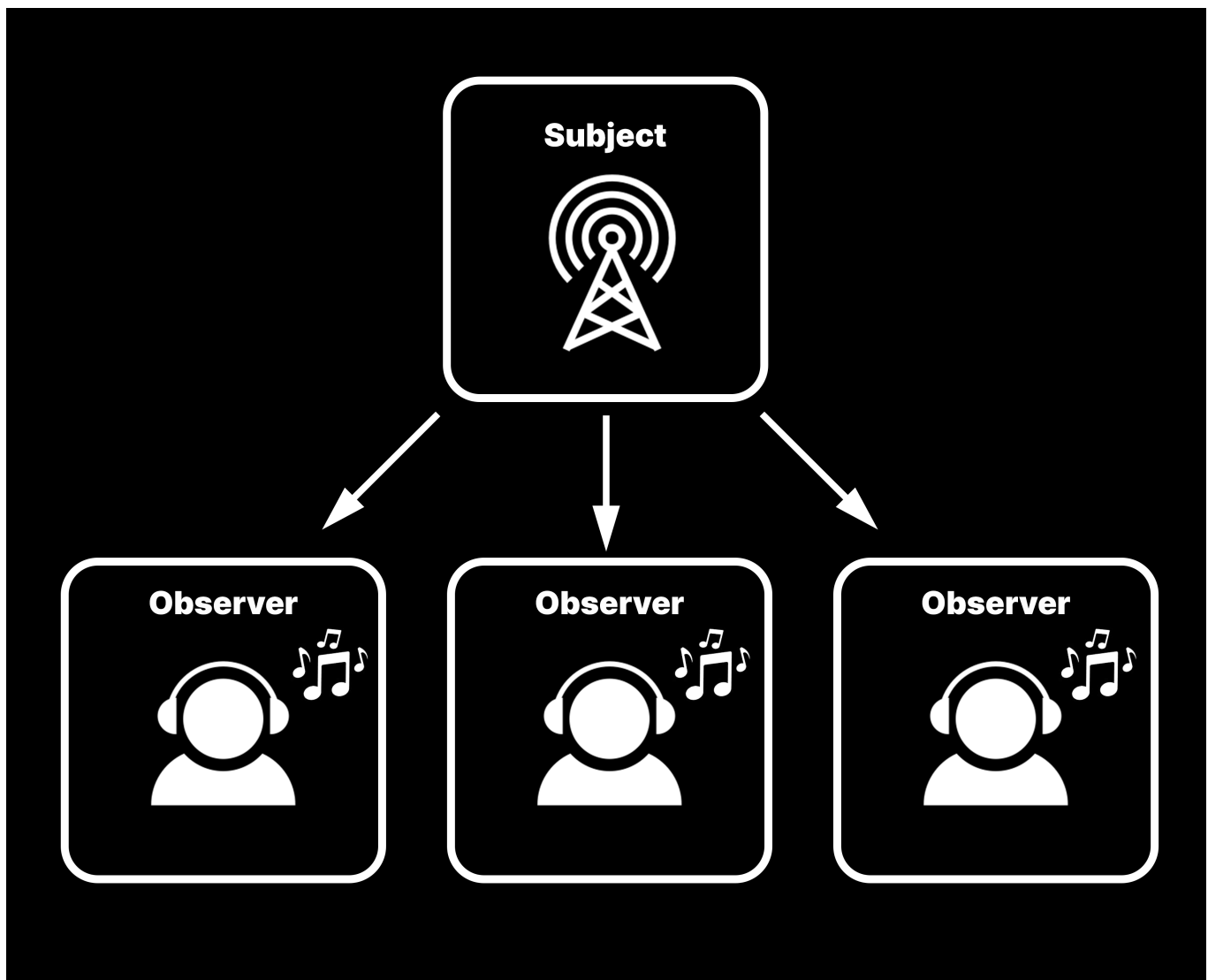
If you do, it will lead to too many dependencies.

- Adds Inflexibility.
- Hard to maintain.

The observer pattern is a common solution to this problem.

"One-to-many" dependency.

When one object changes states, all dependent objects get notified automatically.



The subject is decoupled. It doesn't care about observers.

The observers have a dependency to the subject.

The observers have no dependency on each other.

Anything that raises the Event is acting as a subject.

Anything that responds to the event is an observer.

Different components on the same GameObject can be subjects or observers.

The same component can be a subject in one context, and an observer in another.

Understanding Events

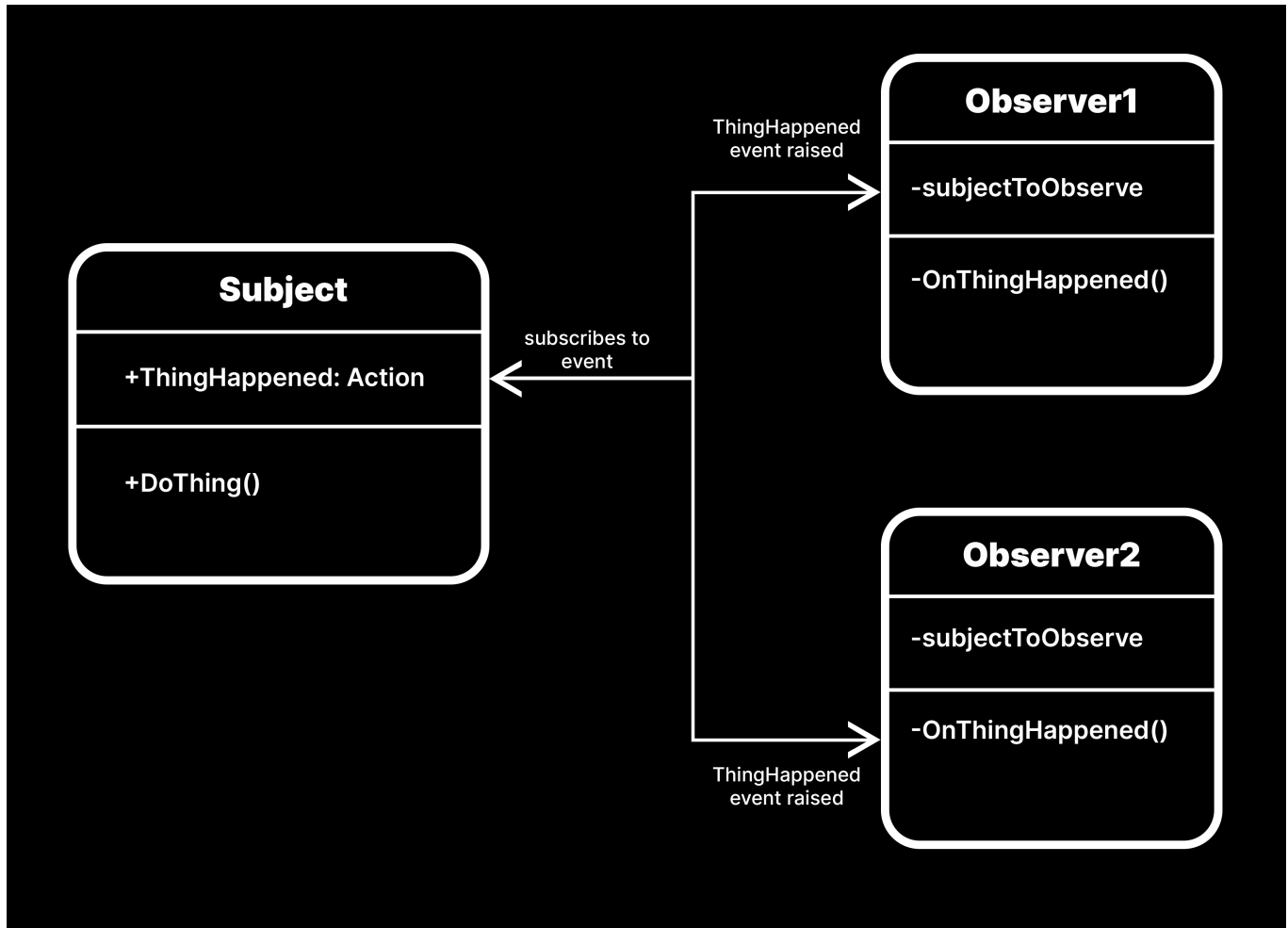
You can design your own subject-observer classes, but C# already implements it using events.

An event is a special type of delegate that allows classes to communicate in a loosely coupled way.

An event is a notification that something happened.

1. A delegate exists. (That's a variable that contains a function.)

2. The subject (publisher) creates an event based on that delegate.
3. The event is just an action the subject takes at runtime. (take damage, or click button etc)
4. Subject maintains a list of observers.
5. Observers subscribe to the subject's event. (Delegate = ThingHappened, Event Handler = OnThingHappened)
6. Event triggers -> Changes state of Subject -> Subject sends notifications to observers.
7. Observers' event handlers get invoked!



Syntax

Subject

```

using UnityEngine;
using System;

// "MonoBehaviour" helps make it easier to attach to a GameObject
public class Subject: MonoBehaviour
{
    public event Action ThingHappened;
}
  
```

```

    public void DoThing()
    {
        ThingHappened?.Invoke();
    }
}

```

Or you can pass parameters using `System.Action<T>` with `List<T>`. Don't know how though.

```

using UnityEngine;
using System;

// "MonoBehaviour" helps mak it easier to attach to a GameObject
public class Subject: MonoBehaviour
{
    public event Action<List<Params?>> ThingHappened;

    public void DoThing()
    {
        // ? means it is only invoked if not null.
        ThingHappened?.Invoke();
    }
}

```

Observer

```

public class Observer : MonoBehaviour
{
    [SerializeField] private Subject subjectToObserve;

    private void OnThingHappened()
    {
        // do stuff
        Debug.Log("Observer responds");
    }

    private void Awake()
    {
        // Subscribe to Subject's Delegate by adding our own function to it
        if (subjectToObserve != null)
        {
            subjectToObserve.ThingHappened += OnThingHappened;
        }
    }

    private void OnDestroy()

```

```

    {
        // To prevent errors if this observer is gone.
        if (subjectToObserve != null)
        {
            subjectToObserve.ThingHappened += OnThingHappened;
        }
    }
}

```

More Improvements to the Pattern

ObservableCollection

- When the list is updated (add, remove, refresh) your observers get notified.

Unique Instance ID as Argument

- If you don't want to trigger every GameObject that shares an observer...
- GameObjects in heirarchy have unique Instance IDs
- You can pass a unique ID into the event (type `Action<int>`)
- Then only run logic in event handler if GameObject matches that unique ID

Static EventManager

- Many Unity apps use a static / singleton EventManager
- Your observers can reference a central source of game events as the subject